

General information

Title : Kitchen LED project overview

Eng : Michael

Change sheet

When?	What?
09/13/2025	Created the document, filled in basic info. Concept description and initial analysis.
09/13/2025	Implement system requirements, freeze the config.
09/14/2025	Change header.

References

WS2811 RGB LED reference

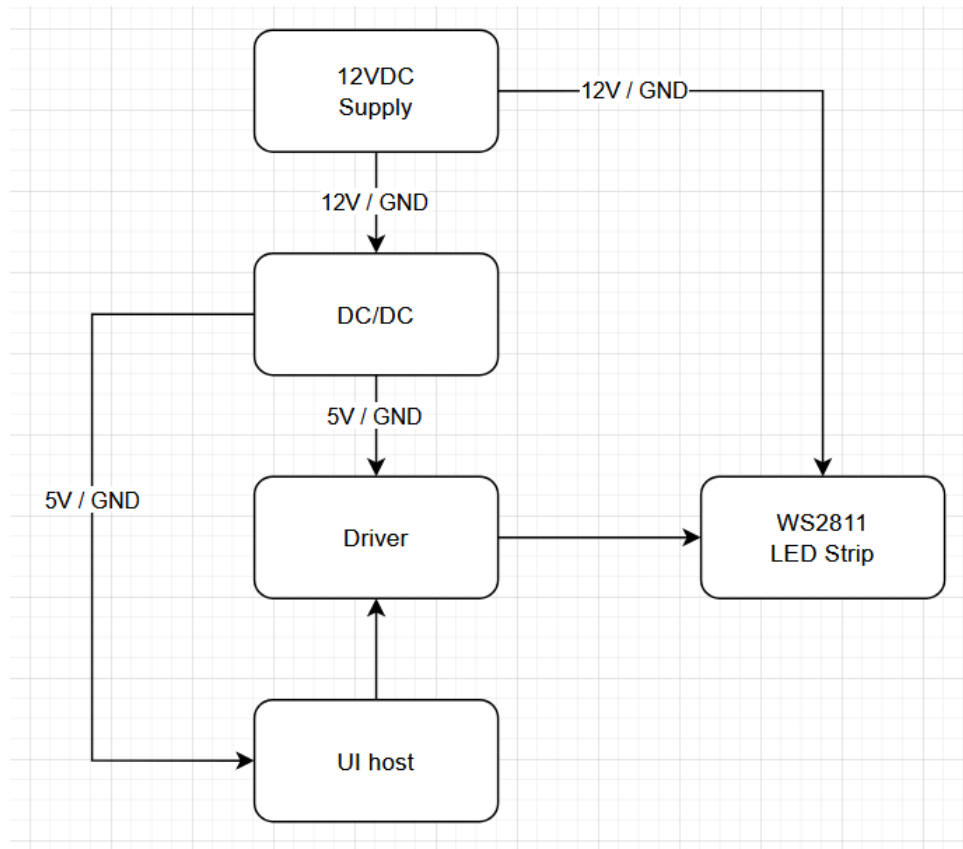
<https://www.tme.eu/Document/26d574b43ad9ddaffa4d5bcd140ec145/WS2811.pdf>

General information	1
Change sheet	1
References	1
Description	3
Block schematic - initial idea	3
Initial analysis	4
Power Supply	4
DC/DC converter	4
Driver and UI host	5
System requirements	6

Description

The aim of this project is to implement a kitchen LED driver that enables to either set working parameters of a **WS2811 RGB LED** manually or over a simple webpage.

Block schematic - initial idea



- 12VDC supply - LED strip requires 12VDC power supply, it's a master source for the whole diagram
- DC/DC - DC converter that enables supplying power to the logic
- Driver - performs low-level operations like transmitting data to LED Strip over SPI / Output Capture / PWM / anything that enables precise timing
- UI host - device that hosts webpage, connected to the driver, can exchange data and force animation change, enable user to implement custom animations

NOTE: If applicable, driver and UI host blocks can be placed on one board, inside one system - separate blocks are used to depict different responsibilities.

NOTE: The driver block should be designed in accordance with WS2811 timing and power requirements. However, with minor adjustments (e.g., replacing the driver block and adapting the power supply), the system can be extended to support other LED types.

Initial analysis

Power Supply

12VDC master power supply - this document assumes use of 5m LED strip, 144LEDs/m.

- Estimated power consumed by strip in the brightest point: 15A
- Estimated power required (25% margin) $\geq 19A$

NOTE: Power that is needed for logic supply is included in the **power required**.

DC/DC converter

If the driver is implemented as a separate device, the DC/DC converter can be integrated into the driver module.

If the driver and UI host are combined in a single system, the use of an external switching regulator is mandatory to step down the 12 V supply to the logic voltage (e.g., 5 V or 3.3 V).

Additional notes:

NOTE: The converter should provide stable output voltage with sufficient current for all logic devices, including MCU, driver, and any peripheral modules.

NOTE: Switching regulators are preferred over linear regulators due to efficiency, especially when driving high-power LED strips and avoiding excessive heat dissipation.

Driver and UI host

As per component's description, **WS2811** requires strict low-level timing.

- **ESP32** - enables driving the LED strip directly (Output Compare/PWM/SPI) and hosting the webpage that allows changing working parameters.

Advantages

- Huge community, lots of available examples and libraries
- Cheap
- Along rich low-level peripheral equipment, WiFi and BT on-board - could run driver and UI host on one board

Drawbacks

- Non-deterministic timing: Many libraries rely on blocking delays (e.g., Delay_ms) which can interfere with other parts of the system
- Although cheap in hardware, the platform may require significant time to learn library behavior or develop custom timing-critical implementations from scratch
- Hard to debug
- **STM32** - enables driving the LED strip directly (Output Compare/PWM/SPI), some of the families allow hosting web server, ethernet or USB connection.

Advantages

- STM32 HAL - huge support from the manufacturer - low-level enclosed in deterministic (at least well tested) abstraction - focus on creating the application
- Cheap
- Allows real-time operation - deterministic behavior
- Easier to debug compared to ESP32 in timing-critical applications

Drawbacks:

- Require more knowledge and experience

- **SBC (Single Board Computer; i.e. raspberry pi)** - runs linux, can host webpage and communicate with hardware.

Advantages

- Huge community
- Lots of tested and supported drivers and packages
- Allows developing the project further - huge flexibility, less constraints than ESP32
- Great debugging tools (memory, debugger and so on)

Drawbacks

- Redundant
- Requires more knowledge and understanding in field of Linux
- More expensive than ESP32

Final Call: The project will be based on deterministic blocks. While this approach may incur higher costs in terms of time and components, it ensures that the resulting system is reliable, predictable, and easier to reuse, adjust, and expand for future improvements or different LED types. Makes debugging and maintenance more straightforward. Enhances scalability and flexibility, allowing integration of new features, alternative LED strips, or additional control interfaces with minimal redesign.

System requirements

This section contains details regarding system requirements - general description of expected results.

NOTE: Hardware/software description files requirements should be based on this.

REQ_ID	Description
SYS_1	Driver and UI host should be separate devices.
SYS_2	Communication protocol should be implemented to exchange data/instructions between the devices - should be a handshake type to enable flexibility and error management.
SYS_3	As per the driver is a separate device from the UI host, it should introduce a power regulator.
SYS_4	The driver should allow changing strip colour, brightness and allow switching supply for LED Strip on/off.
SYS_5	Blocks server and driver , should allow communication over a popular hardware peripheral. UART is a baseline that should satisfy this requirement.
SYS_6	Server block API should be available at a static IP (baseline). Access over a local DNS system would be a great solution.
SYS_7	The server should offer a driver baseline (changing strip colour, brightness and allow switching supply for LED Strip on/off).
SYS_8	The server should host a form that enables custom animation creation.